Contents

Abstract

The **Temperature Sensor Project (with ESP32 and ThingSpeak)** is an Internet of Things (IoT) based system designed to monitor ambient temperature and humidity and perform automated control actions when predefined thresholds are crossed. The system uses an ESP32 microcontroller interfaced with a DHT series sensor (DHT11/DHT22) to collect environmental data, transmits the readings to ThingSpeak for cloud logging and visualization, and triggers actuators (such as a fan or LED) based on configurable temperature thresholds. This project demonstrates real-time monitoring, remote data logging, basic control logic for automation, and practical insights into embedded IoT development using the Arduino ecosystem.

1 Introduction

Environmental monitoring is essential in a variety of contexts: indoor climate control, greenhouses, server rooms, and simple consumer devices. Manual monitoring is time-consuming and error-prone. IoT-based monitoring enables automated, continuous observation and action. This project demonstrates a compact, cost-effective implementation of temperature and humidity monitoring with cloud logging and simple automatic control using the ESP32 microcontroller and ThingSpeak as the cloud platform.

Project Objectives

- Design and implement a temperature and humidity measurement system using ESP32 and DHT sensor.
- Send sensor readings to ThingSpeak for cloud storage and visualization.
- Implement threshold-based actuator control (fan/LED) locally using GPIO or via remote commands.
- Provide clear documentation, code, and placeholders to include diagrams and dashboard screenshots.

2 System Components and Requirements

2.1 Hardware Components

- **ESP32 Development Board** — central microcontroller with integrated Wi-Fi for cloud connectivity.
- **DHT22 (preferable) or DHT11** — digital temperature and humidity sensor. DHT22 offers higher accuracy and wider range.
- **Relay Module** (optional) — for switching higher-power devices (fans) using the ESP32 GPIO.
- **Mini Fan or LED** — actuator to demonstrate control logic. Use a fan behind a transistor/relay when using real motors.
- **Power Supply** — USB 5V for ESP32; separate 5V/12V supply for fan if required by current draw.
- **Connecting Wires, Breadboard / PCB** — to prototype the circuit.
- **Optional: Level shifter** — if using 5V logic devices with strict voltage requirements.

Sensor: DHT22 vs DHT11

- **DHT11:** Cheaper, lower precision, narrower temperature range (0–50°C), lower humidity accuracy.

- **DHT22:** Better precision, wider temperature range (-40–80°C), better humidity accuracy. Recommended for more reliable results.

2.2 Software Components and Services

- **Arduino IDE / PlatformIO** — to write and upload firmware to ESP32.
- **ESP32 Arduino Core** — libraries providing WiFi and hardware support.
- **DHT sensor library** — to interface with the DHT22/11 sensor.
- **ThingSpeak IoT Platform** — cloud channel for data logging and visualization (fields for temperature, humidity and status).
- **WiFi Network** — reliable internet connectivity for the ESP32 to transmit data to ThingSpeak.
- **Optional: MQTT Broker or HTTP APIs** — as alternatives for cloud upload or command/control.

3 System Architecture

3.1 Overall Architecture

The system follows a typical IoT flow:

1. Sensors (DHT22) measure temperature and humidity.
2. ESP32 reads sensor values, applies smoothing/calibration, and decides whether an actuator should be toggled.
3. Data is sent to ThingSpeak for storage and real-time visualization using HTTP / ThingSpeak library.
4. The actuator (fan/LED) is driven directly via GPIO (or via relay module for higher voltage/current loads).
5. A web dashboard on ThingSpeak displays charts and logs; alerts can be created based on thresholds.

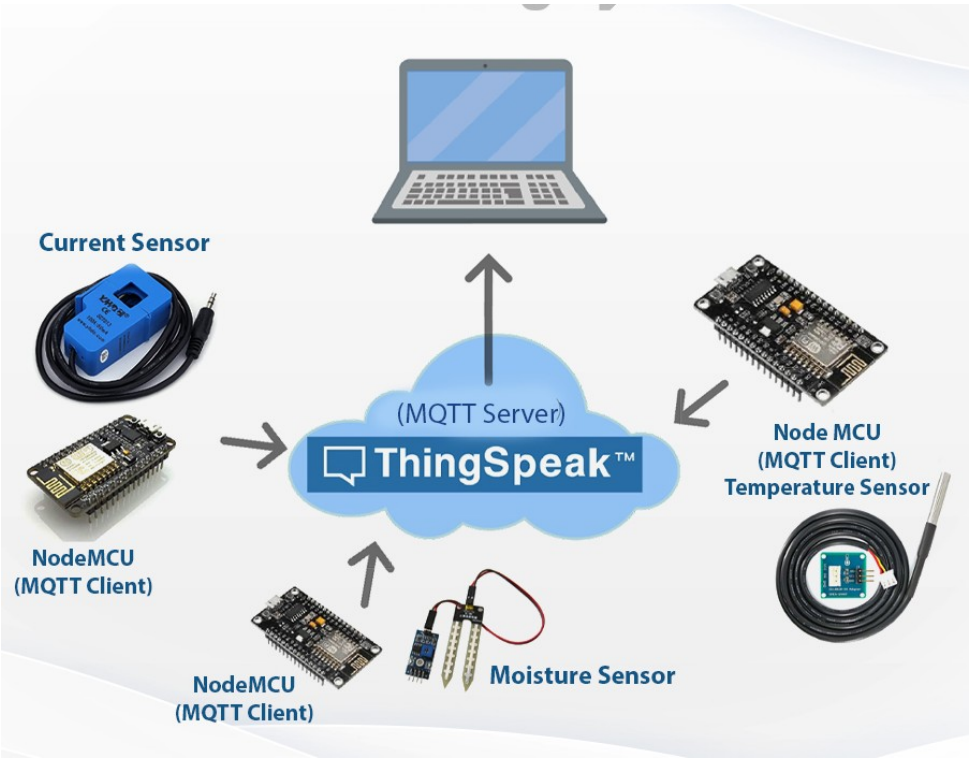

Figure 1: High-level system architecture

3.2 Hardware Setup (Wiring)

- **DHT22:**
  VCC → ESP32 3.3V
  GND → ESP32 GND
  DATA → ESP32 GPIO (e.g., GPIO4) (use a 10k pull-up resistor between DATA and VCC if required)
- **Fan / Relay:**
  Relay IN → ESP32 control pin (e.g., GPIO12)
  Relay VCC → 5V supply
  Relay GND → common GND
  Fan power supply connected through relay COM/NO contacts (keep ESP32 GND common with relay GND)

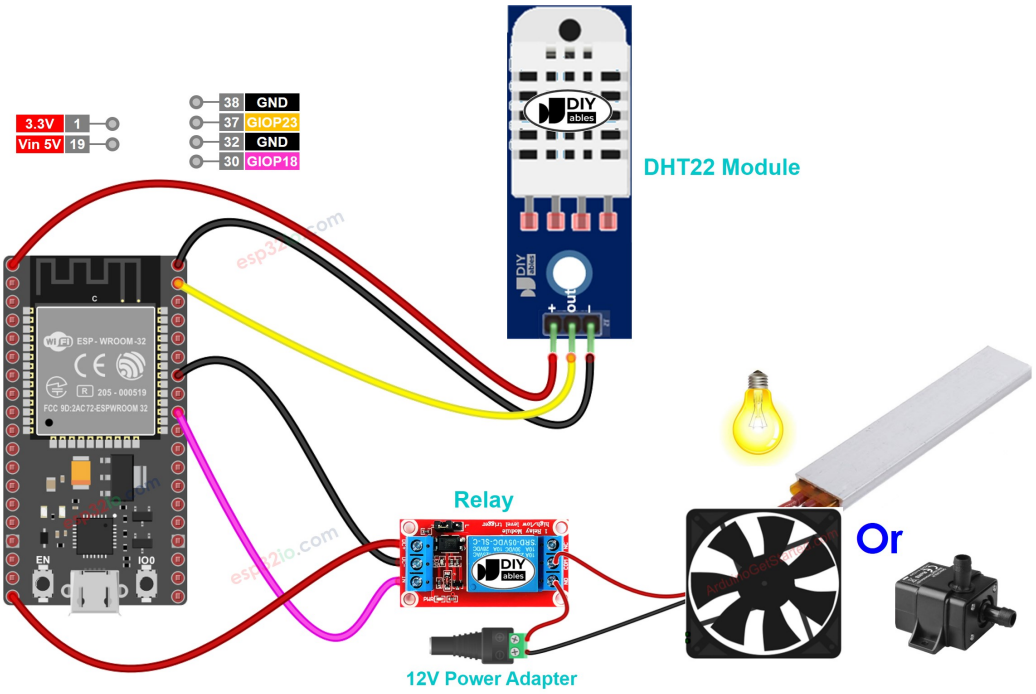

Figure 2: Typical wiring between ESP32, DHT sensor and relay/fan

3.3 Software Flow

1. Initialize serial, Wi-Fi, ThingSpeak client, and sensor.
2. Read temperature and humidity at fixed intervals (e.g., every 10–30 seconds).
3. Validate readings (check for `NaN`), apply optional smoothing (moving average).
4. Send values to ThingSpeak fields (temperature, humidity, actuator status).
5. Compare temperature against thresholds; if exceeded, switch actuator ON; else OFF.
6. Optional: Provide HTTP fallback or retry logic on failed uploads.

4 Code Implementation

This section provides complete firmware code (Arduino style) for the ESP32 with comments and explanations. The code:

- Connects to Wi-Fi,
- Reads DHT sensor values,
- Uploads readings to ThingSpeak,
- Implements actuator control logic,
- Includes retry and basic error handling.

4.1 Required Libraries

- `WiFi.h` — for Wi-Fi connectivity.

- `ThingSpeak.h` — ThingSpeak client library.
- `DHT.h` — sensor driver for DHT11/22.

4.2 Complete Arduino/ESP32 Code

Listing 1: ESP32 code: DHT22 + ThingSpeak + actuator control

```
/*
  Temperature Sensor Project (ESP32 + DHT22 + ThingSpeak)
  - Reads temperature & humidity
  - Uploads to ThingSpeak
  - Controls an actuator (fan/LED) based on temperature threshold
*/

// Include libraries
#include <WiFi.h>
#include <ThingSpeak.h>
#include "DHT.h"

// ========== User configuration ==========
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASSWORD";

unsigned long channelID = YOUR_CHANNEL_ID;        // ThingSpeak channel number (unsigned long)
const char* writeAPIKey = "YOUR_WRITE_API_KEY";   // ThingSpeak write API key

#define DHTPIN 4            // GPIO connected to DHT data pin
#define DHTTYPE DHT22       // DHT 22 (AM2302); change to DHT11 if used

DHT dht(DHTPIN, DHTTYPE);
WiFiClient client;

// Actuator pins
const int ACTUATOR_PIN = 12;   // Relay or transistor control pin (active HIGH)
const float TEMP_THRESHOLD = 30.0; // Temperature in degree Celsius to enable actuator

// ThingSpeak timing constraints
const unsigned long UPDATE_INTERVAL_MS = 20000; // 20 seconds (respect ThingSpeak rate limits)
unsigned long lastUploadTime = 0;

void setup() {
  Serial.begin(115200);
```

```
  delay(100);

  pinMode(ACTUATOR_PIN, OUTPUT);
  digitalWrite(ACTUATOR_PIN, LOW); // Ensure actuator off at start

  // Initialize sensor
  dht.begin();

  // Connect to Wi-Fi
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  unsigned long wifiTimeout = millis() + 15000;
  while (WiFi.status() != WL_CONNECTED && millis() < wifiTimeout) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("WiFi connected");
    ThingSpeak.begin(client);
  } else {
    Serial.println("WiFi connection failed - will attempt later");
    // Depending on needs, you can implement reconnect attempts in loop
       ↪ ()
  }
}

void loop() {
  unsigned long now = millis();

  // Read sensor values
  float temperature = dht.readTemperature(); // Celsius
  float humidity = dht.readHumidity();

  // Validate readings
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
    delay(2000);
    return;
  }

  // Debug print
  Serial.print("Temperature: "); Serial.print(temperature); Serial.print
     ↪ ("  C , ");
  Serial.print("Humidity: "); Serial.print(humidity); Serial.println("
     ↪ %");
```

```
80
81    // Actuator logic: simple threshold-based control
82    if (temperature >= TEMP_THRESHOLD) {
83      digitalWrite(ACTUATOR_PIN, HIGH); // Turn ON fan/LED
84      Serial.println("Actuator: ON");
85    } else {
86      digitalWrite(ACTUATOR_PIN, LOW); // Turn OFF
87      Serial.println("Actuator: OFF");
88    }
89
90    // Upload to ThingSpeak (respecting rate limits)
91    if (now - lastUploadTime >= UPDATE_INTERVAL_MS) {
92      if (WiFi.status() != WL_CONNECTED) {
93        // Try to reconnect
94        Serial.println("WiFi not connected. Attempting reconnect...");
95        WiFi.begin(ssid, password);
96        unsigned long wto = millis() + 10000;
97        while (WiFi.status() != WL_CONNECTED && millis() < wto) {
98          delay(500);
99          Serial.print(".");
100       }
101       Serial.println();
102     }
103
104     if (WiFi.status() == WL_CONNECTED) {
105       ThingSpeak.setField(1, temperature);
106       ThingSpeak.setField(2, humidity);
107       ThingSpeak.setField(3, digitalRead(ACTUATOR_PIN)); // actuator
            ↪ status
108       int resp = ThingSpeak.writeFields(channelID, writeAPIKey);
109       if (resp == 200) {
110         Serial.println("Successfully uploaded to ThingSpeak");
111       } else {
112         Serial.print("ThingSpeak write failed, response = ");
113         Serial.println(resp);
114       }
115     } else {
116       Serial.println("WiFi still not connected; skipped ThingSpeak
            ↪ upload");
117     }
118     lastUploadTime = now;
119   }
120
121   // Delay between sensor reads
122   delay(2000);
123 }
```

4.3 Explanation of Key Code Sections

Wi-Fi Initialization: The ESP32 attempts to connect to the configured Wi-Fi. A timeout is applied to avoid indefinite blocking in `setup()`.

DHT Readings: `dht.readTemperature()` and `dht.readHumidity()` return float values. Always check for `NaN` to detect sensor read errors.

Actuator Control: A simple threshold comparison toggles the actuator GPIO. For real motors, always use a relay or MOSFET with proper flyback/protection and a separate power supply if required.

ThingSpeak Uploads: Use the official ThingSpeak library to set fields and write fields. Respect the minimal interval policy of ThingSpeak (update interval typically not shorter than 15 seconds for free channels).

Error Handling: The sketch includes basic reconnection attempts and prints response codes to Serial for debugging.

5 Smoothing and Calibration

Sensor readings can be noisy. Two common approaches:

- **Moving average:** Maintain a small circular buffer (e.g., last 5 readings) and compute mean before uploading/control.
- **Offset calibration:** If sensor shows systematic bias, apply an offset or scaling factor determined experimentally.

6 Security Considerations

- Avoid hardcoding Wi-Fi credentials and API keys in public repositories. Use a separate header file (`secrets.h`) ignored by VCS, or use OTA/secure storage options.
- ThingSpeak write API key is sensitive — keep it private.
- For production, prefer TLS-based cloud endpoints or MQTT with proper authentication.

7 System Testing and Results

7.1 Testing Strategy

- **Unit testing:** Verify raw sensor output via Serial Monitor.
- **Integration testing:** Check that ThingSpeak receives data and that the graph updates.

- **Actuator testing:** Test the relay and fan with safety precautions (no bare hands near spinning fan).
- **Edge cases:** Simulate very high/low temperatures and intermittent Wi-Fi outages to verify failover behavior.

7.2 Expected Dashboard / Visualizations

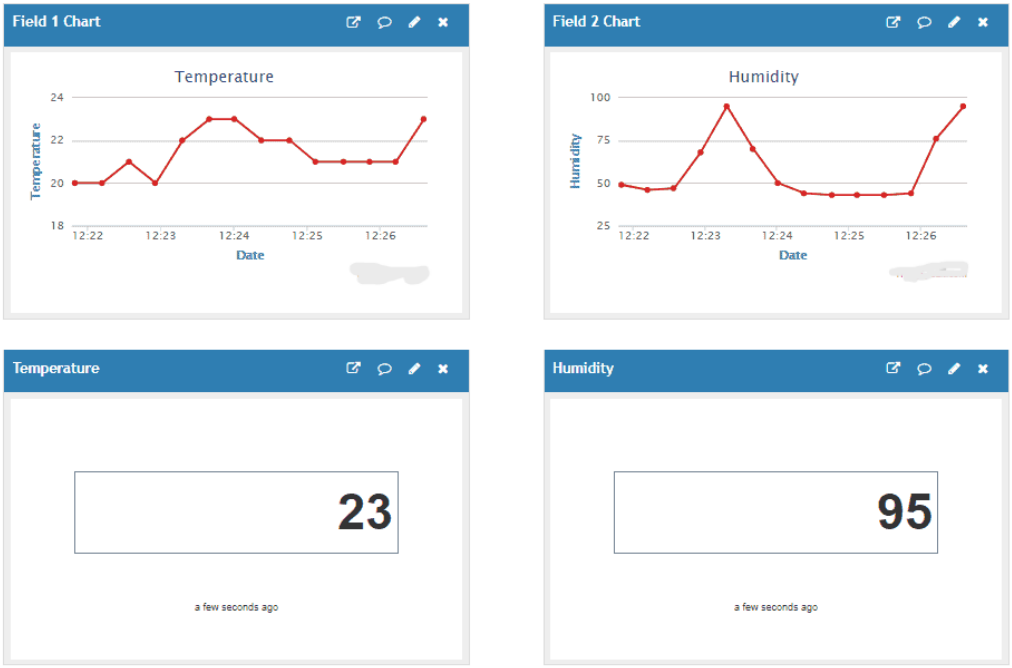

Figure 3: ThingSpeak channel visualization (temperature and humidity fields)

8 Future Scope

The project provides a foundation for many enhancements:

- **Multiple Sensor Nodes:** Extend to multiple ESP32 nodes distributed across rooms/greenhouse zones.
- **MQTT Integration:** Use MQTT with TLS for more flexible pub/sub architecture and bi-directional control.
- **Mobile App / Web Dashboard:** Create a custom dashboard or mobile app for improved UX and notifications.
- **Machine Learning:** Use historical data to predict temperature/humidity trends and perform predictive control.
- **Power Optimization:** Implement deep sleep and wake-up strategies for battery-operated sensors.
- **Additional Sensors:** Add CO_2, light (lux), or gas sensors and expand to full environmental monitoring.

- **Secure OTA Updates:** Implement secure Over-The-Air updates to push firmware patches safely.

9 Conclusion

This Temperature Sensor Project demonstrates a simple yet practical IoT application: collecting temperature and humidity data using a DHT sensor and an ESP32, transmitting the readings to ThingSpeak, and performing threshold-based control of an actuator. The implementation highlights important aspects of IoT systems: sensor interfacing, data validation, cloud connectivity, actuator control, and basic reliability measures. The modular design allows straightforward expansion to multi-node setups, improved analytics, and more secure communication for production deployments.

Bibliography

- ESP32 Official Documentation — Espressif. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/
- Arduino Reference and Libraries — Arduino. https://www.arduino.cc/reference/en/
- ThingSpeak Documentation — MathWorks. https://thingspeak.com/docs
- DHT22 Sensor Datasheet and Tutorials (Adafruit / Various tutorials)
- PubSubClient / MQTT Libraries (if using MQTT alternatives)

10 Bill of Materials (BOM)

Item	Quantity	Remarks
ESP32 Development Board	1	Any dev board with USB interface
DHT22 Sensor	1	Recommended; DHT11 can be used for lower accuracy
Relay Module (5V)	1	For fan/motor control
Mini Fan or LED	1	Actuator
Jumper wires	Several	For prototyping
Breadboard	1	Optional
Power supply (5V/1A)	1	For ESP32 and relay

Table 1: Estimated Bill of Materials

11 Quick Setup Checklist

1. Assemble hardware: wire DHT sensor, relay and fan according to schematic.
2. Insert Wi-Fi credentials and ThingSpeak channel/key into the code.
3. Upload sketch to ESP32 using Arduino IDE / PlatformIO.
4. Open Serial Monitor (115200) to observe debug prints.
5. Confirm ThingSpeak channel receives readings and graphs update.
6. Test threshold-based actuator behavior by changing ambient temperature (or adjust TEMP_THRESHOLD temporarily).